

Tablo Adı	Türkçe Karşılığı	İşlevi
Users	Kullanıcılar	Genel kullanıcılarımız
OperationClaims	Yetkiler	Yetki kısıtlamaları olacak o yetkiye sahip olmayan kişinin erişim/işlev/kontrol izni olmayacak
UsersOperationClaims	Kullanıcı Yetkileri	Kullanıcılar ile yetkileri ilişkilendireceğiz
Companies	Şirketler	Şirketlerin tutulduğu tablo
AccountReconciliations	Hesap Mutabakatları	
AccountReconciliationDetails	Hesap Mutabakat Detayları	01.01.2021 ile 31.12.2021 arasında bir hesap mutabakatı yapmak istiyorsak bu mutabakatın detaylarının ekstresini sisteme yükleyebileceğim. Bu sayede kullanıcı hem istediği bakiyeyi hem de detay listesini görebilecek. Bu sayede detay listesine göre isterse şu kısımda bir hata/yanlışlık var gibi bir mesaj gönderebilecek.
BaBsReconciliations	BaBs Mutabakatları	
BaBsReconciliationDetails	BaBs Mutabakat Detayları	
UserCompanies	Kullanıcı Şirketleri	
CurrencyAccounts	Cari Firmalar	
Currencies	Döviz Cinsleri	Dolar Euro TL gibi hespi bu tabloda bu yüzden yapı bozulmadan tek yerden sabit çekiliyor. 0 1
MailParameters	Mail Parametreleri	Otomatik mail gönderimi

Users Tablosu Database

Id
Name
Email
PasswordSalt
PasswordHash
AddedAt (kullanıcı oluşturma tarihi)
IsActive (kullanıcı şu an aktif mi)
MailConfirm (kullanıcı kaydı oluşturduğunda mail onayı göndereceğiz mail onayı dönerse onu orada tutacaz
MailConfirmValue (mail gittikten sonra adam o maile tıklarsa doğrularsa kontrol eder tekrardan mail gönderilirse tekrardan uniqueidentifier benzersiz oluştururuz.
MailConfirmDate (adam kullanıcı oluşturmuş ama mail gitmedi adam da geldi mail onayı butonundan tekrardan bir mail onayı isteyecek bu mail onayı butonunu manipüle edemesin yani arka arkaya her butona tıkladığında sürekli mail gitmesin her 10 da bir sadece bir mail atabilsin.

OperationClaims Tablosu	
Id	(Yetkilerin Tutulduğu Tablo) (Ekleme, çıkarma, Ba, Bs, gibi)
Name	(her şeye dair bir yetki vereceğiz sonra frontend kısmında kısıtlama yaptırabileceğiz)
AddetAt	(ekleme zamanı)
IsActive	(Hala aktif olup olmadığı) (Aktif değilse listede bize getirme gibi)

UsersOperationClaims	
Id	(Bizim ilişki tablomuz)
UserId	(User ile operationclaims tablolarını birleştireceğiz)
OperationClaimId	
AddedAt	
IsActive	

Companies	
Id	
Name	
Address	
TaxDepartment	(vergi dairesi)
TaxIdNumber	(vergi numarası) (boş olabilir çünkü bir firmanın vergi nosu olabilir tc nosu olabilir veya ikiside olmayabilir)
IdentityNumber	(tc numarası)
AddedAt	
IsActive	

UserCompanies	
Id	(İlişki Tablomuz User ile Company Tablolarını ilişkilendirecek)
UserId	
CompanyId	
AddedAt	
IsActive	

CurrencyAccounts	
Id	(Bu tablo bizim Carilerimiz)
CompanyId	(Koyma sebebimiz bu cari hangi şirket açmış bilmemiz gerekiyor ki ona göre kayı tutulsun)
Code	(Koyulma sebebi kullandığınız programda, netsis, logo, zirve hatta excel takibi bile yapsanız farketmez buna bir cari kod veriyoruzdur mutlaka biz bu kodları alalım mi tekrardan cari eklemeye çalıştığımızda bu kodlarla biz bunu konte(sayma) edelim ya da tam tersi işlem yapalım. Mesela sizin kullandığınız programdan biz buna veri aktarmak istersek onunla kodları eşletirebilelim diye bir ön hazırlık yapalım)
Name	
Address	
TaxDepartment	(vergi dairemiz null olabilir)
TaxIdNumber	(vergi numarası null olabilir)
IdentityNumber	(null olabilir)
Email	(firmaya mail atabilmek için firmanın mailini kaydetmemiz gerekiyor) (null olabilir çünkü maili olmayan bir firmayı da mutabakat yapabilelim. Sözlü mutabakat yapabiliriz. Dönüp programdan sözlü mutabakatı işaretleriz)
Authorized	(yetkili kişi manasında kullanıldı ama bizim burada kullandığımız maksat bizim buradaki görüştüğümüz kişi mailde olsa telefonda da olsa authhorized.)
AddedAt	
IsActive	

Currencies	
Id	(para birimlerini tutacağız. Türk lirası, dolar, euro, sterlin...)
Code	(Bu kısımda biz Angular da, tablelerimizde falan para cinsini gösterebilmek için kodlarını kullanacağız yani doların USD, türk lirasının TL gibi)
Name	(Bu kısma direkt uzun açıklamasını yazacağız. Türk lirası, dolar, sterlin..)

AccountReconciliations	
Id	
CompanyId	(ŞirketId)
CurrencyAccountId	(CarId)
StartingDate	(Bu bir mutabakat olacağı için tarih aralıkları girerek gönderebiliriz)
EndingDate	
CurrencyId	(Paramızın ID si)
CurrencyDebit	(Debit ve Credit kısmı borç alacak kısım)
CurrencyCredit	
IsSendEmail	(Mail gönderildi mi manasında)
SendEmailDate	(Mail gönderdiğimiz tarihi saklayacak)
IsEmailRead	(Biz bir link göndereceğiz o linke tıkladığı zaman otomatik olarak kaydedeceğiz.)
EmailReadDate	(Siteye ulaştığı tarihi kaydedecek)
IsResultSucceed	(Sonuç true ise başarılı yani onaylanmış)
ResultDate	(Sonucun tarihi)
ResultNote	(Sonuçla alakalı not) (Olumsuz bir dönüş yaptıysa mesela mutabakat değiliz dediyse buraya nedenini falan yazabilir gibisinden)

AccountReconciliationDetails
Id
AccountnReconciliationId
Date (Mesela 100 tl bize borcunuz var deniyor bu da bu 100 tl yi detaylandırıyor)
Description (Bu belgenin ne olduğuna dair)
CurrencyId (Para cinsi)
CurrencyDebit
CurrencyCredit

BaBsReconciliations
Id
CompanyId
CurrencyAccountId
Type (Ba, Bs yi temsil etmesi için)
Mounth
Year
Quantity (Adet manasında)
Total
IsSendEmail
SendEmailDate
IsEmailRead
EmailReadDate
IsResultSucced
ResultDate
ResultNote

BaBsReconciliationDetails
Id
BaBsReconciliationId
Date (Faturaları tarihi)
Description (Faturaların açıklaması)
Amount (Faturanın KDV hariç tutarı)

MailParamaters
Id
CompanyId
Email
Password (Mailimizin şifresi)
SMTP (Mail göndereceğimiz sunucu adresi)
Port
SSL (True False tutacağız)

Cari Bilgi: muhasebenin iki tarafı arasında ilerleyen alacak verecek işlemlerinin hesaplarını, bu hesaplara dair bilgi.

E-Mutabakat, beyanname alış (BA), beyanname satış (BS), cari hesap mutabakatları ve ekstre karşılaştırma işlemlerini faks ya da telefona gerek topluca yapılmasını sağlayan sistemdir? e eder.

Beyanname, bir kişinin veya bir kurumun vergi ödemesi yapması gereken miktarı belirlemek için kullanılan bir belgedir.

Beyan öncesi ve beyan sonrası dönemler olmak üzere ikiye ayrılır.

- Beyan öncesi dönemde, vergi ödemesi yapılması gereken miktarın hesaplanması için gerekli bilgiler toplanır ve beyan öncesi beyanname düzenlenir.
- Beyan sonrası dönemde ise, vergi ödemesi yapılmış ve beyan öncesi beyanname doğrulanmış olur. Beyan sonrası dönemde, vergi ödemesi yapılacak miktarın hesaplanması için gerekli bilgiler toplanır ve beyan sonrası beyanname düzenlenir.

Mutabakat nedir?

Şirketler arası yada kişiler arası bazı konularda ortak nokta bulunmasına denir. Bunlar alacak verecek yada yıllık izin gibi konularda gerçekleşebilir. Şirketler genelde yıl sonu yaklaştığında her alanda kişilerle yada diğer firmalarla mutabık olmak isterler. Çalışanların yıllık izinleri de buna dahildir. Yıllık izinleri ile de anlaşma sağlanması gerekir. Aksi takdirde işçinin hesapladığı gün sayısı ile iş verenin hesapladığı gün sayısı arasında fark oluşabilir. Bu gibi durumlar zaman içerisinde sorunlara yol açabilir.

Cari mutabakat genelde yıl sonlarına doğru yapılır. İki şirketin de sisteminde aynı alacak verecek rakamları var ise kendi aralarında mutabık olurlar.

Database Oluřturma

New Database

Select a page

- General
- Options
- Filegroups

Script Help

Database name: eReconciliationDb

Owner: <default>

☒ Use full-text indexing

Database files:

Logical Na...	File Type	Filegroup	Initial Size (...)	Autogrowth / Maxsize	Path
eReconciliation...	ROWS ...	PRIMARY	8	By 64 MB, Unlimited	C:\Progra...
eReconciliation...	LOG	Not Applicable	8	By 64 MB, Unlimited	C:\Progra...

Connection

Server: DESKTOP-TI9JN12

Connection: DESKTOP-TI9JN12\\rumey

[View connection properties](#)

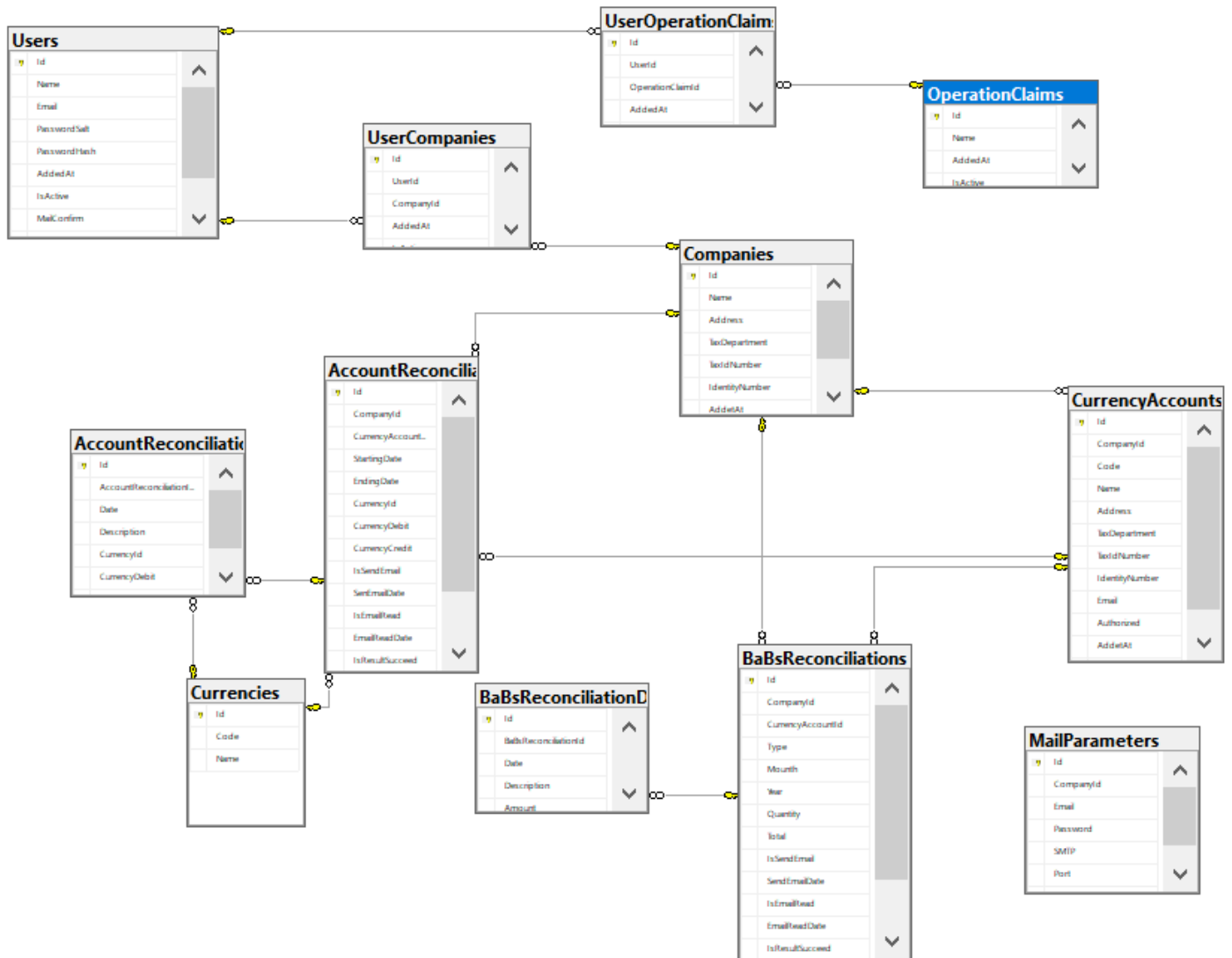
Progress

Ready

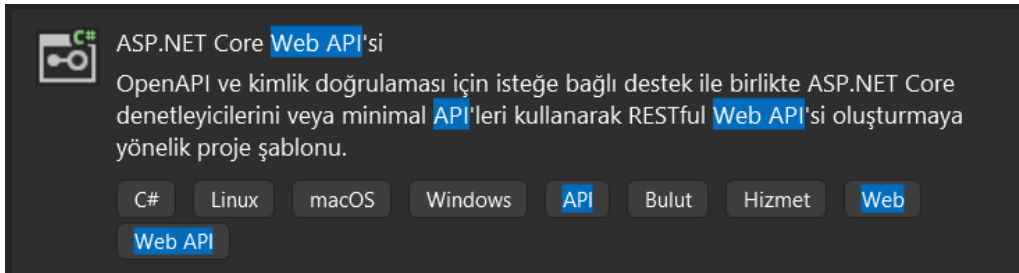
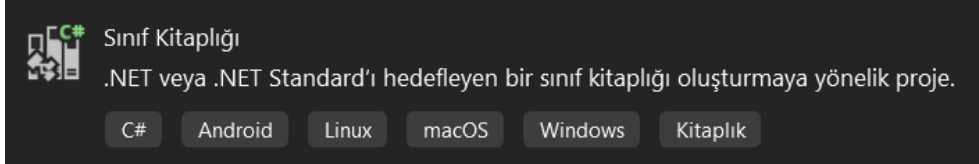
Add Remove

OK Cancel

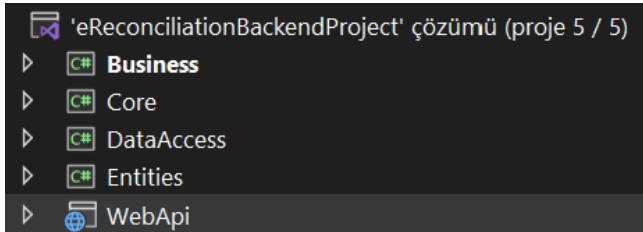
PassWordSalt ve PassWordHash bizim řifremizi biz arkaplanda kriptoloyacađız yani řifremizi tekrardan bir řifreleyeceđiz ki bunu millet burada goremessin. Yarın öbür gün database imizi patlatırlarsa buradan girip kullanıcı ve řifreleri alamasınlar manasında o yüzden biz bunu iki farklı řifreleme metoduyla harmanlayıp kriptolama yöntemi uygulayacađız. Çünkü tek Hashleme kullanırsak sağlıklı deđil hash i kırabiliyorlar bunun için araçlar var ama salt ile beraber kullanıldığında çok sağlam bir yapı oluyor.



PROJE OLUŞTURMA

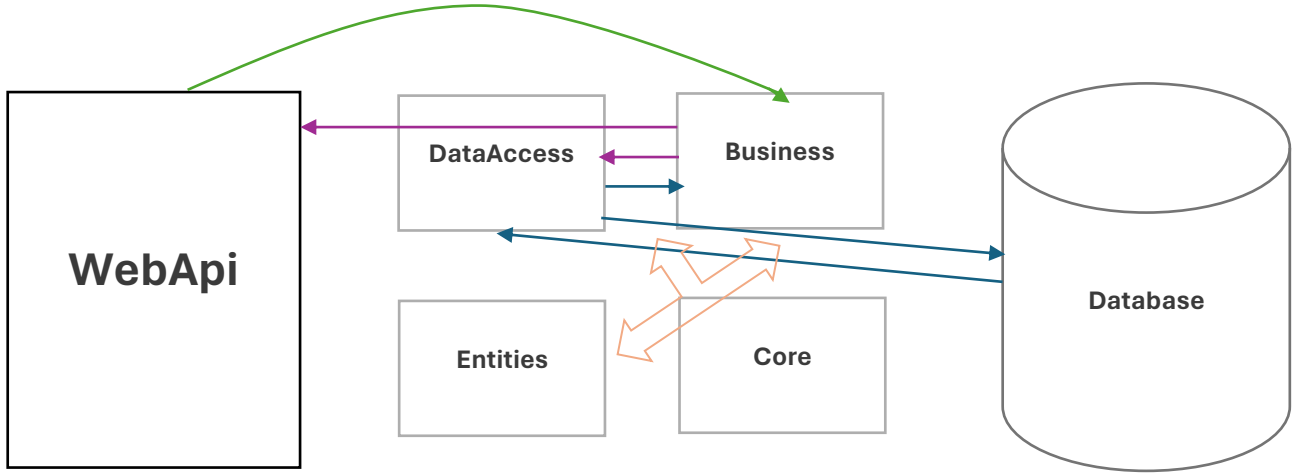


Katmanlı Mimarimiz



Katmanlı Mimari nedir? Bizim burada her bir katmanın kendine ait bir işi yapması ve bu sayede tek bir katman üzerinde tüm işlemlerin yapılabildiği bir karmaşanın oluşmasını engellemek için çıkmıştır.

Neden ihtiyaç duyuyoruz? Şu anda bizim burada mevcut bir projeyle site yapıyoruz. Bizim bu sitede işlemlere ihtiyacımız olacak. En basitinden create, update, delete, read (ekleme, güncelleme, silme, okuma) işlemleri yapacağız. (işlem gönderiyorum, işlem alıyorum, işlem gönderiyorum alıyorum.....) Bu işlemler için bazı kontroller gerçekleştireceğiz. Validation kontrol gibi Validation kontrolünde de mesela ben bir mutabakat eklemek istiyorsam örnek veriyorum 300 TL'nin altındakileri cari seçilmemişse mutabakat listesine ekleme, cari oluştururken mesela diyeceğiz ki cari ismi 3 karakterden az olanların ismini oluşturma bunlara işte Validation kontrol deniyor. İşte şimdi bu tür işlemleri eğer katmanlı mimari oluşturmazsam WebApi kısmının Controllers'ında tek tek yazmam gerekecek kod kalabalığı oluşacak elimizde bir karmaşa oluşacak. Katmanlı Mimari işte bizi bu kalabalıktan kurtarıyor.



Ben bir kayıt işlemi yapmak istersem bunu direkt gidip business katmanına gönderiyorum. Business katmanım bu kayıt için gerekli olan validation, transection, loglama gibi özellikleri konte ediyor eğer herhangi bir sıkıntı yoksa DataAccess Katmanına gidiyor. DataAccess katmanı artık kendi olan tek görev bu dataları işlemektir. Hemen gidiyor Database'e işlemi yaptırıyor, sonucunu alıyor, Business katmanına geri dönüyor. Business katmanı sonuçta eğer hiçbir sıkıntı bilmazsa WebApi'ye cevabını gönderiyor. Böyle WebApi katmanımız tertemiz oldu çünkü her katman kendine ait işi yapıyor.

Entities katmanı bizim classlarımızı, tablolarımızı tutuyor. Core katmanı ise tüm projeye hizmet eden ortak yapıları tutuyor. Mesela Validation için bir Validation tool yazacağız Core katmanı tutacak ve Business katmanına gönderecek. DataAccess katmanı için, Entities katmanı için, WebApi katmanı için başka bir özellikler tutacak. Kısaca Core katmanı hizmet katmanıdır. Tüm projede kullanabileceğimiz özellikleri yazarız sonra Core katmanına atarız Core katmanı da tüm projeye hizmet eder.

Her proje, her katman kendine ait bir işi yapıyor, başka bir iş yapmıyor. Bu sayede bir hata alsak veya işlemi irdelenmek istesek hangi katmana bakacağımızı ve nasıl yapacağımızı rahat bulabiliriz.

Entities Katmanı

Class'ların Oluşturulması

Abstract, soyut nesneleri tutmak için. Concrete somut nesneleri tutmak için. Biz burada classlar oluşturacağız. Classları da Concrete katmanına ekleyeceğiz.

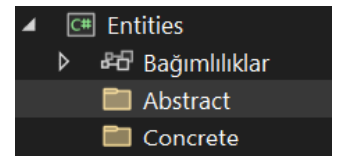
Bunları yapmadan önce ilk bir projemizde SQL çağırıyoruz.

Database de money olarak geçen veri tipinin karşılığı kodlamada decimal.

İlk olarak AccountReconciliationDetail içerisini doldurduk. Daha sonra gittik Core içerisine Entities diye bir klasör ekledik. Klasör içerisine de bir IEntity adından interfaces(arabirim) ekledik. Bu boş kalıyor.

DateTime? Burada ? koymamızın anlamı bu verim boş olabilir. Null işaretlediklerimizde koymamız gerekiyor. Eğer koymazsak program hata verir.

!!!!!!11.derste 3.dakikada SenEmail ile SendEmail arasında bir yanlışlık var ilerde hata alırsan dön bak



OperationClaim, UserCompany, UserOperationClaim, User tablolarını Core katmanında Concrete klasöründe oluşturuyoruz. Bunun da sebebi biz Core katmanında bunlarla alakalı işlemlerde yapacağız ve Userlar burada olması gerekiyor. Userla alakalı kullanıcılarla işlemler Core katmanında tokenlarla kontrol eteceğiz token kodlayacağız.

CodeFirst Yaklaşımı => Visual Studio üzerinden SQL bağlantısı yapıp tekrardan gidip SQL'e veritabanı oluştur arkasından gel tekrardan VisualStudio da oluştur gibisinden değil de Direkt tüm işlemlerimizi VisualStudio üzerinde yapıyoruz. VisualStudio otomatik gidip oluşturduğumuz tüm table(sınıfları) gidiyor SQL'de oluşturuyor. CodeFirst yaklaşımının temel mantığı da VisualStudio'da tüm tablelerimizi oluşturduk daha sonra DataAccess katmanında bağlantı kurduk sonrada VisualStudio üzerinden Package Manage Console dan komutlarla database oluşturabiliyoruz. Ama biz bu yapıyı kullanmadık. Bunun sebepleri de birincisi burada eğer bu yapıyı kullanıyorsak Attributeleri da burada tanımlamamız gerekir. Bu CodeFirst yaklaşımını kullanırsak bu bundan sonra projeye bağımlı kalıyor yani sen proje üzerinde yanlışlıkla bu yapıyı bozacak bir şey yaptın mesela database yerini değiştirdin veya table nin yapısını değiştirdin ve güncellemeyi unuttun programı çalıştırmaya başladığında program sana şunu demeye başlıyor ya senin database yapınla programda göndermeye çalıştığın yapı tutumuyor beni eşleştir diyor çünkü biz asıl databaseimizi VS üzerinde oluşturduk SQL de değil. Artık bunları eşleştirmek için kodlar yazmamız gerekiyor ve bazen bu kodlar çalışmıyor. İnternette cevabını bulamadığımız kodlar da oluyor. Bu yüzden bu yapıyı kullanmadık. Zaten classların içine Attribute yazmak da sıkıntı orda gidip key tanımlaman validationları yapman, alanları belirlemen alanlardan kastımız veri tipinde varchar(200) falan belirlediğimiz 200. Böyle Böyle kod karmaşası da oluşuyor.

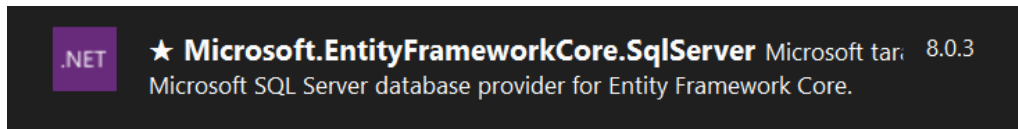
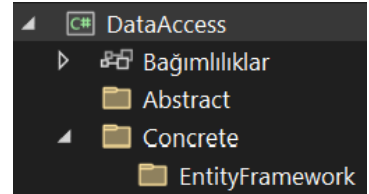
[] içine yazdıklarımıza Attribute deniyor. Örneğin [Key].

DataAccess Katmanı

Abstract soyut, Concrete somut nesneler için.

Bizim burada Database bağlantımızı yapmamız lazım. Concrete klasörüne EntityFramework adında bir klasör daha ekliyoruz.

ORM bizim VisualStudio muz ile veritabanımız arasında bağlantı yapabilen ara programlardır. Bu sayede bizim kodlamamızı SQL diliyle veya başka dillerle yapmıyoruzda direkt ORM in kendi araçlarıyla yapıyoruz. EntityFramework bunlardan biri ve çok kullanışlı bir araç. Bu sayede SQL ile veri çekeceksem veya sorgulayacaksam EntityFramework un direkt kendi kodlama metotlarını kullanır. Bunun için öncelikle bir paket kurmamız gerekiyor. Bunun için DataAcces katmanımıza sağ tıkladık ve Manage NuGet Packages(NuGet Paketlerini yönet) tıklıyoruz. Browse(Gözet) kısmına geliyoruz.



Bunu kurduktan (6.0.3versiyon) sonra EntityFrameWork içine Context dosyası oluşturuyoruz. Klasöre EntityFramework denmesinin de sebebi kullandığımız ORM teknolojisinin adını yazıyoruz ki yarın öbür gün başka bir ORM e geçersen direkt onunla klasörlendirip onunla çalışabileyim diye.

Context içine de ContextDb adında bir sınıf ekliyoruz. Buraya SQL bağlantımız için gerekli kod bloklarını ekleyeceğiz.

```
0 bagvuru
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(@"Server=DESKTOP-TI9JN12;Database=eReconciliationDb;Integrated Security=true");
    //Integrated Security bizim bağlantımızın güvenli olup olmadığını sorguluyor. Bunu da biz lokal makinelerde kullanıyoruz. Lokal ile true yapıyoruz.
}
```

```
optionsBuilder.UseSqlServer(@"Server=DESKTOPTI9JN12;Database=eReconciliationDb;Integrated Security=true");
```

Integrated Security bizim bağlantımızın güvenli olup olmadığını sorguluyor. Bunu da biz lokal makinelerde kullanıyoruz. Lokal ile true yapıyoruz. Kullanıcı bilgileri yazsaydık false yapıcaktık.

Daha sonra ContextDb içine classlarımızın isimlerini tanımlayacağım.

CRUD (Create – Read – Update – Delete) İşlemleri Nedir? Temelde database de tablelerde yaptığımız tüm işlemlerin kısaltması. Bunları biz DataAccess katmanında tanımlarız. Business katmanında ise bu işlemleri çağırarak öncesini ve sonrasını kontrolleri veya işlemleri yazıyoruz. Validation ile kontrol et transection ile kontrol et gibi.

Abstractlarda interfacesler bizim imza tutucularımızdır. İçlerinde sadece metotların imzaları bulunur.

Bunun için DataAcces katmanına bir ilk önce ICompanyDal isimli bir interfaces ekledim. Bunun içine artık Company için ne işlem yapmak istiyorsam (CRUD işlemleri yani) onları yazıyoruz. İçine yazdığımız crud işlemlerini generic yapıdan dolayı direkt çağırıyoruz.

Generic Yapı nedir? Temel mantıkları şimdi biz ilk Company e ait temel seviye CRUD işlemleri oluşturduk artık elimizde bizim bir yapımız var şimdi bizim 12 tane table miz var 12 tane classım var yani. Bu 12 tan class için bu işlemleri tek tek interfaces olarak oluşturmak hepsine tek tek girip imzalarını oluşturmak istemiyoruz. İşte burada Generic Yapılar devreye giriyor. Burda şunu yapıyoruz o zaman crud işlemlerini bir defaya mahsus generic yapıda oluşturuyorum sonra o generic yapıya diyoruz ki sen ebnim seni çağıracağım yerdeki class ı al bu sayede bana bir daha bu yapı tekrar tekrar oluşturtma. Generic yapı Core Katmanında oluşuyor. Core katmanına gidiğ DataAccess adında bir folder oluşturuyoruz sonra o folder a bir IEntityRepository adında interfaces ekliyoruz.

Normalde biz `void Add(Company company);` yazardık ama şimdi

Şimdi `void Add(T entity);`

Burada bizim T yi kullanabilmemiz için public yanına başa aşağıdaki kodu ekliyoruz

```
public interface IEntityRepository<T> where T : class, IEntity, new()
```

Bunun açıklaması diyoruz ki IEntityRepository bir tane T al bu T de class olmalı, IEntity ile implant edilmeli yani IEntity buna eklenmiş olmalı. Aynı zamanda da new lenebilir olsun. Yani ben buraya interfacesler gönderemeyim onlar newlenemez ama classlar newlenebilir.

Artık biz bunu bir Generic yapıda oluşturmaya başladık.

```
List<T> GetList(Expression<Func<T,bool>>filter = null);
```

Burada biz ben bir sorgu yapabilirim bu sorgum aynı zamanda null da olabilir. İki durumda da bunu çalıştır dedik.

Daha sonra gittik Core katmanı içerisine bir DataAccess klasörü oluşturduk. O klasörün içine de : IEntityRepositoryBase sınıfı ekledik. Bu sını içerisinde Tentity ve Tcontextlerimizi ayarladık.

Artık önceden CRUD işlemleri için oluşturduğumuz IcompanyDal içine tek tek CRUDları yazmak yerine artık saece public yanına : **IEntityRepository<Company>** yazıyoruz ve böylelikle tek tek yazmaktan kurtulup tanımladıklarımızı yani imzalarımızı tek bu kodla çağırış oluyoruz.
Şimdi DataAccess deki Concrete içerisinde ki EntityFramework dosyamıza EfCompanyDal isimli bir sınıf ekliyoruz.

Daha sonra her sınıfa Abstract içine IAccountReconciliationDetailDal gibi interfacesler oluşturuyoruz. İçerisine **public interface**
IAccountReconciliationDetailDal:IEntityRepository<AccountReconciliationDetail>

Daha sonra IAccountReconciliationDal interfaces ekliyoruz.

public interface IAccountReconciliationDal : IEntityRepository<AccountReconciliation>

IBaBsReconciliationDal ekliyoruz. Özetle hepsine adım adım IEntityRepository eklemesini DataAccess de Abstract içerisine açıyoruz.

Biz şimdi bu adımlar ile generic yapımızı anladık ve oluşturduk artık sıra generic yapıyı classlara uygulamak. Bunun içinde generic yapımızın concrete kalsörünü tamamlayacağız. DataAcces katmanımızın Concrete içindeki EntityFramework içerisine class oluşturup uygulayacağız. Bunun içinde önceki adımlar gibi her sınıfı isimlendireceğiz bu sefer I yerine hepsinin başına Ef eki ekliyoruz ve bunları IEntityRepository yerine EfEntityRepositoryBase eklemesi ve ContextDb den veri çekmesi için ekleme gerçekleştiriyoruz ardından soyut interfacemizi ekliyoruz birinin örneğini vericek olursam : (Bu işleme implament deniyor.,)

public class EfAccountReconciliationDal :
EfEntityRepositoryBase<AccountReconciliation,ContextDb> , IAccountReconciliationDal

Bunu her sınıfa uygulayacağız. Böylece Generic yapımızı tamamlamış olacağız.

Business Katmanı

Business Katmanına da ilk önce Abstract ve Concrete olmak üzere iki klasör eklemesi gerçekleştirdik. Daha sonra Abstract kısmına bütün sınıflarımız için birer ICompanyService gibi interfaces eklemesi gerçekleştirdik. Bunları implament etmeyeceğiz metod imzaları ile public yapılarımızın içlerini dolduracağız. Business Katmanında DataAccess katmanında yaptığımız CRUD işlemlerinden ihtiyacımız olanları ekleyeceğiz.

Şirket listelemek için ICompanyService de public içerisine `List<Company> GetList();` kodunu ekledik. Böylece tüm şirketimizin listesini şu an da bu kod ile çağırmış olduk.

Daha sonra Concrete klasörü içerisine CompanyManager adında bir class eklemesi gerçekleştirdik. İçerisine de ICompanyService implamentini yani arabirim(interfaces) uygulamasını gerçekleştirdik.

Peki neden DataAccess katmanına yaptığımız şeyleri burada da yapıyoruz? Bunun sebebi biz burada çağırdığımız metota yetkilendirme yapacağım, validation kontrol yapacağım, transection yapacağım vb. Yani biz buradaki class a diyeceğiz ki bu işlemi yapmadan önce kontrol et mesela bu kullanıcı yetkili mi diye kontrol et yetkiliyse o zaman transaption gerektiren bir durum var mı onu kontrol et. Bu işlemi log dosyasına kfydet. Bu işlemin Validation kontrol kısımları var ise Validationlarını kontrol et. Eğer bunlardan hiçbirinde sıkıntı yoksa o zaman dön bu işlemim yap. İşte Business katmanını bunun için kullanıyoruz.

Bu işlemlerin benzerini ve kontrolünü yaptırmak için Business katmanına kodlama yapıyoruz:

- Kullanıcı Yetkili mi?
- Trancaption gerektiren durum var mı?
- Log kaydet,
- Validation kontrol.

DataAcces katmanında genel yapıyı oluştururken Business katmanında işlemin artık kalan kısımlarını tamamlıyoruz. Bu sayede de bizim WebApi katmanında sadece Business katmanındaki `GetList();` metodunu çağırıyoruz. Başka hiçbir kodlama yapmıyoruz. Çünkü o metodun içerisinde artık her şey olmuş oluyor.

Concrete içerisinde `GetList();` için public içerisine bir şeyler çağırmamız gerekiyor. Bunun için kodlama gerçekleştiriyoruz.

```
EfCompanyDal efCompanyDal= new EfCompanyDal();
```

Bu kod ile yapıyı bağımlı halde getirdik. Bizim bu kurumsal mimaride işimizi zorlaştırır. Yarın öbür gün projeyi farklı ortamda yapmak istersek yapımızı tamamen değiştirmemiz gerekir.

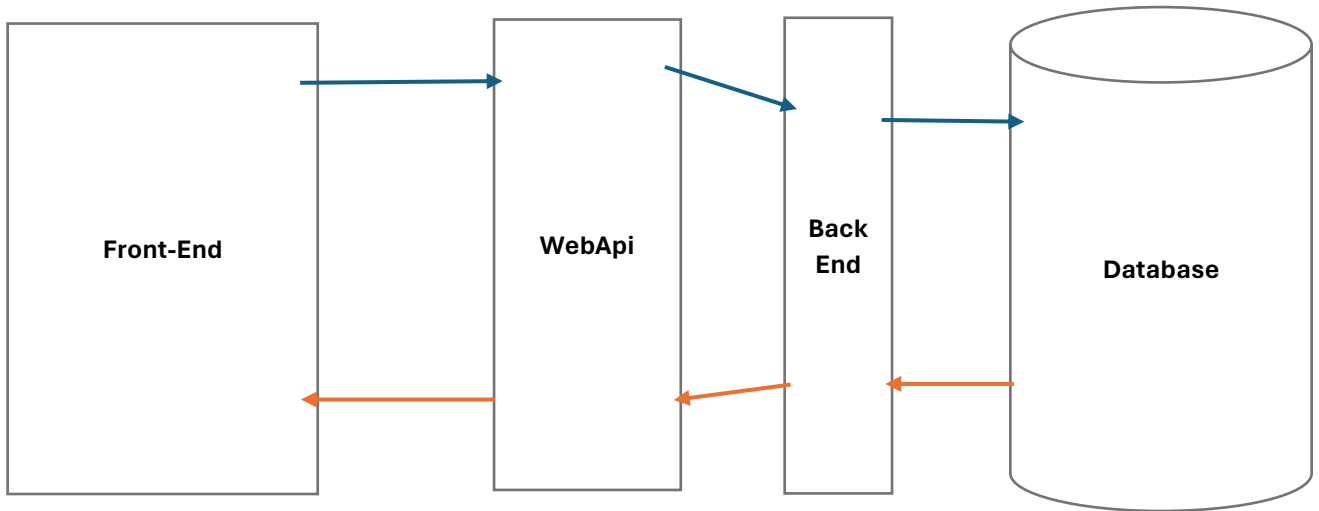
Bunun için devreye Dependeceny Injection giriyor.

Dependeceny Injection bağımlılık injeksiyonu demek. Soyut somut bağılılığı. bizim somut bağılılığımızı yani biz EntityFramework e bir somut bağılılık yaptık. Bunu soyut bağılılığa çeviriyor. Yani biz yine orada bir bağlantı yapacağız ama bağlantı soyut olduğu için somut kısmı değiştirebilecek.

Yarın öbür gün mesela Oracle geçmek istersek sadece ICompanyDal → OcCompanyDal diye eşleştirme yapacağız. MySql e çevirirsek de MyCompanyDal diyeceğiz gibi.

Yani o zaman biz bir önceki kodda EfCompanyDal yerine ICompanyDal kullanacağız. Bu sayede EntityFramework e bağılılıktan kurtulmuş olduk.

```
public class CompanyManager : ICompanyService
{
    private readonly ICompanyDal _companyDal;
    public CompanyManager(ICompanyDal companyDal) //Interfaces newlenemediği için
newleme işlemi yapıyor.
    {
        _companyDal = companyDal;
    }
    public List<Company> GetList()
    {
        return _companyDal.GetList();
    }
}
```



Biz Front-End den bir veri istediğimizde ilk önce WebApi katmanına gidiyor. WebApi bunu Back-End e iletiyor. Back-End de alıp bunu Database e iletiyor. Database de aynı sırada Front-End e dönüyor.

Eğer biz Select * From Companies dersek bu sırada bir sonuç dönüyor o sonuç da liste şeklinde geri dönüyor.

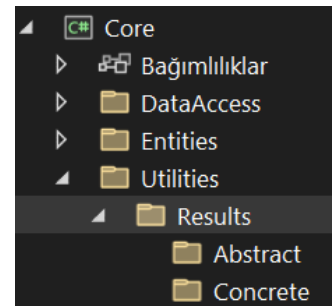
Eğer Insert Into Companies dersek de sadece bir ekleme işlemi yapar bir sonuç dönmez.

İşlem Sonuç Sistemi : Herhangi birinde işlem yaptığımız zaman hangi işlemi yaptığımızın bir önemi yok. Bu sistem bize desin ki senin bu işlemin başarılı, başarısız al istediğin data al istediğin veri. Bana her yaptığım işlemde istisnasız hangi işlemi yaparsam yapayım bana bu sonucu göndersin. İşte biz buna işlem sonuç sistemi diyoruz. İşlem sonuç sistemizde bir result yapısı oluşturacağız(core katmanına). Bu oluşturduğumuz yapı sonucunda ben hangi veriyi istersem isteyeyim yani hangi sorgu ya da işlemi yaparsam yapayım bu bana sonuç olarak bunlardan (başarılı,başarısız,data,veri) gönderecek. Yani WebApi kısmından Front-End e verdiği veri kısmından bu işlemin başarılı olup olmadığı sonuç olarak bir mesaj eklemek istersem içerisine bir mesaj olup olmayacağı ve en sonda içerisinde bir veri yan gerekirse bir sorgu yapıyorsam select sorgusu gibi onun verisi olacak.

Result Yapısının Kurulması

Biz bunu Core katmanına kuracağız. Bunu da Core katmanına Utilities klasörü Results klasörü oluşturarak gerçekleştireceğiz. Abstract içerisine IResult adında bir interfaces oluşturuyoruz ve içerisine iki değişken atıyoruz.

```
public interface IResult
{
    public bool Success { get; }
    public string Message { get; }
}
```



Ardından aynı yere IDataResult adında bir interfaces oluşturuyoruz. İçerisinde

```
public interface IDataResult<out T> : IResult
{
    T Data { get; }
}
```

Şuan interfaces yapılarımızı tamam sıra class yapılarımıza geldi. Bunu da Concrete klasörü içerisine ekleyeceğiz. Burayada Result ve DataResult olarak iki sınıf oluşturduk. Burada şu işlemi yaptık. Bir işlem gönderdiğimde bana bir sonuç gönder. Sonucumuzda işlemin ve mesajın başarılı olup olmadığı. Sonuç ve mesajın ayrı ayrı oluşmasının sebebi de ayrı ayrı isteyebiliriz.

```
public class Result : IResult
{
    public Result(bool success)
    {
        Success = success;
    }
    public Result(bool success, string message) : this(success)
    {
        Message = message;
    }

    public bool Success { get; }
    public string Message { get; }
}

}
```

DataResult: Burada artık bir veri istediğimde data mesaj ve durumu gelecek ya da sadece data ve durumu gelecek. Buna sonuç yapısı diyoruz.

```
public class DataResult<T> : Result, IDataResult<T>
{
    public DataResult(T data, bool success, string message): base(success, message)
    {
        Data = data;
    }
    public DataResult(T data, bool success): base(success)
    {
        Data=data;
    }

    public T Data { get; }
}
```

Yapiyı daha düzgün hale getirmek için Concrete altına SuccessResult sınıfı ekledik. Bu işlem başarılıysa bana göndereceği veriyi gösterecek. İşlem sonucu döndürecek.

Daha sonrada SuccesDataResult sınıfı ekledik. Bu veri sonucu döndürecek.

Ardında hata döndürebilmesi için ErrorDataResult sınıfını ekledik.

Son olarak da ErrorResult sınıfı ekledik.

Şimdi oluşturduğumuz Result sistemlerini Busniess katmanına oluşturacağız.

Abstract içindeki ICompanyService içerisine IDataResult uyguladık.

```
IDataResult<List<Company>> GetList();
```

Daha sonra Concrete içerisindeki CompanyManager içerisinde önceden yazdığımız return işlemini sildik.

İmlament interfaces yaptık

Daha sonra Business katmanına Constans isimli bir klasör ekledik. Şimdi biz CompanyManager içerisine bir mesaj yazdırmıştık bu mesajı statik hale getirelim yani ordan kaldıralım ve onu da bir yerden alalım diye bu klasörü oluşturduk. Constans klasörü içerisine de Messages isimli bir class ekledik.

```
public static string AddedCompany = "Şirket kaydı başarıyla tamamlandı.";
```

içerisine bu kodu ekledik.

Daha sonra da CompanyManager sınıfımıza dönüp

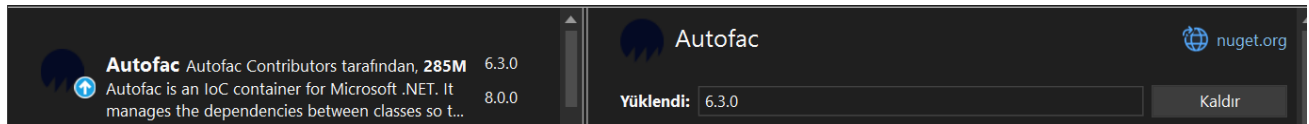
```
public IResult Add(Company company)
{
    _companyDal.Add(company);
    return new SuccessResult(Messages.AddedCompany);
}
```

Yazarak bu olaydan kurtulmuş olduk.

Artık tüm mesajlarımızı Messages sınıfı içerisine yazıp oradan çağırabileceğiz.

Autofac Dependency

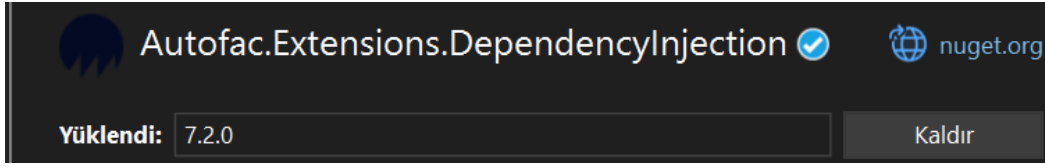
Bunun için Business katmanımıza DependencyResolvers isimli bir folder ekledik. Daha sonra Business katmanında NuGet ile



Kurulumu gerçekleştiriyoruz.

Ardından DependencyResolvers klasörüne Autofac isimli bir klasör ekliyoruz Ardından altına AutofacBusinessModule sınıfını ekliyoruz. Bu sınıf ile biz server ile managerı DataAcces katmanındaki ICompanyDal ve EfCompanyDal ile eşleştireceğiz.

Şimdi WebApi katmanımıza gidiyoruz. Program.cs içerisine Autofac yaptığımız işlemi tanımlayacağız. Bundan önce NuGet ile tekrardan bir eklenti kurmamız gerekiyor.



Kurulumu gerçekleştirdik.

```
builder.Host.UseServiceProviderFactory(new AutofacServiceProviderFactory());  
  
builder.Host.ConfigureContainer<ContainerBuilder>(builder => builder.RegisterModule(new  
AutofacBusinessModule()));
```

Program.cs içerisine bu kodu ekledik. Artık bizim AutofacBusinessModule de oluşturduğumuz yapıyı WebApi katmanımıza dahil ettik.

Business Katmanı Refactory

Business katmanında Abstract içerisine IAccountReconciliationService interfaces ekliyoruz. Concrete içerisine de AccountReconciliationManager sınıfı ekliyoruz. İhtiyaç halinde bunları dolduracağız. Manager içine IAccountReconciliationService dahil ettik. Bunu her class için yapıyoruz.

Bunları her class a yaptıktan sonra dependency injectionlarını yapacağız. Bunun için DependencyResolvers klasörümüzde Autofac içerisindeki AutofacBusinessModule içerisine her class için aşağıdaki kodu ekliyoruz.

```
builder.RegisterType<CompanyManager>().As<ICompanyService>();  
builder.RegisterType<EfCompanyDal>().As<ICompanyDal>();
```

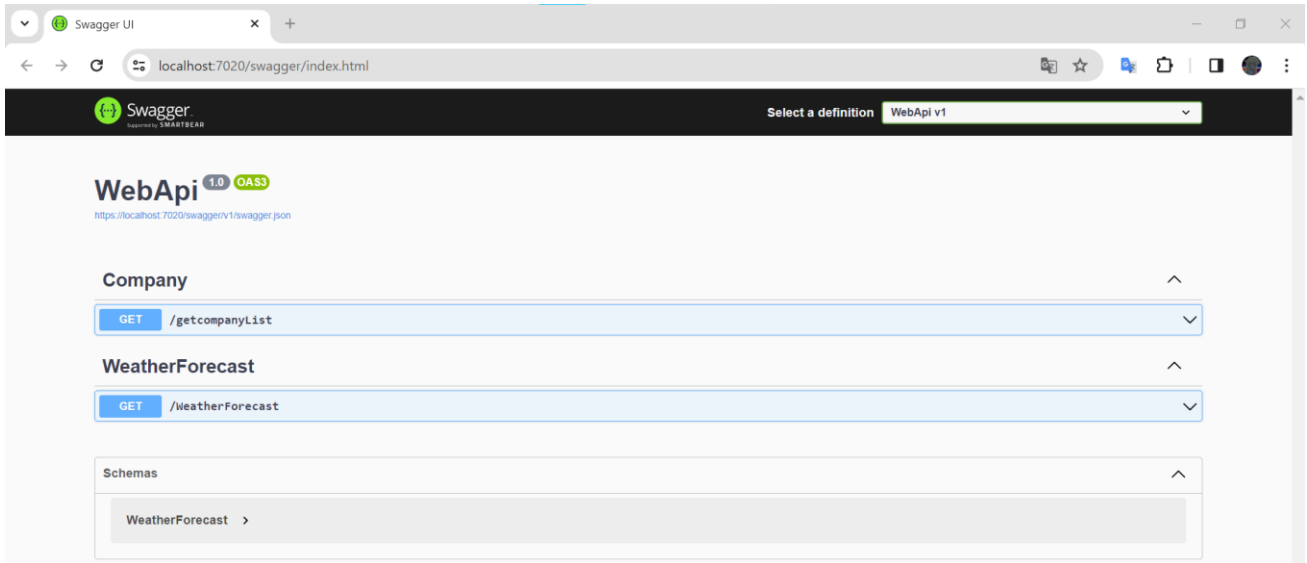
..

WebApi Katmanını Kodlamak

Görsel kısımda neler yaptığımıza ve ne sonuçlar alacağımıza bakmak için WebApi katmanını kullanacağız.

İlk olarak Controllers klasörüne bir adet CompanyController adında (MVC Empty) bir denetleyici ekliyoruz.

WebApi çalıştırdığımızda karşımıza böyle bir site geliyor



Swagger yapısı WebApi'nin özelliklerinden biri.

Rol Bazlı Yapının kurulması

Dto(DataTransferObject) İmplementasyonu, Yetki Listesinin Getirilmesi

Entities katmanına bir Dtos adında klasör ekliyoruz. İçerisine de UserForLogin adında bir class ekliyoruz. Bundan önce de referans alabilmemiz için Core katmanında Entities içerisine IDto adında bir interfaces ekliyoruz. İmplement edeceğiz. UserForLogin içerisine aşağıdaki kodları ekledik.

```
public string Email { get; set; }
public string Password { get; set; }
```

Bir de Dtos altına UserForRegister adında bir class ekliyoruz. İçerisine de ekliyoruz.

```
public string Name { get; set; }
public string Email { get; set; }
public string Password { get; set; }
```

Artık kullanıcının kayıt ve giriş için Dtoları hazır.

Şimdi DataAccessimizdeki IUserDal içerisine Operationclaimleri dahil edeceğiz. Yani biz burada kullanıcıyı kaydettikten sonra bu kullanıcıya yetkiler vereceğiz. Bu yetkiler çerçevesinde bu kullanıcının yetki kontrollerini sorgulamasını sağlamamız lazım. Onun içinde bu kullanıcının yetkilerini bir yerde tutup çekmemiz gerekiyor.

```
List<OperationClaim> GetClaims(User user, int companyId);
```

Kullanıcıların hangi şirkete ait yetkilerinin olduğunu görmek için bu kodu ekledik.

EfUserDal içerisine dönüp

```
using (var context = new ContextDb())
{
    var result = from operationClaim in context.OperationClaims //burada tüm yetkilerin
listesini çekiyorum
                join userOperationClaim in context.UserOperationClaims //bu kod ile
kullanıcı yetkileri table'da bulunan yetkiler ve bir üstteki yetkiler arasında eşleştirme yapıyorum ki, kullanıcı yetkileri
table'da yazdığım Id bilgisinin karşısında name bilgisi çekebileyim.
                on operationClaim.Id equals userOperationClaim.OperationClaimId
                where userOperationClaim.CompanyId == companyId &&
userOperationClaim.UserId == user.Id //burada kullanıcı ve şirket bilgisine göre kısıt veriyorum
                select new OperationClaim
                {
                    Id = operationClaim.Id,
```

```

        Name = operationClaim.Name,
    }; //burada da yeni bir OperationClaim nesnesi türetip içine sadece UserOperationClaim
kısmında yetki evrdiğim kullanıcı ve yetkileri listesini çekiyorum.
    return result.ToList();
}

```

JWT Alt Yapısının Kurulması, Kayıt ve Giriş İşlemleri

Token Yapısının Kurulması

İlk olarak Core katmanımızda Utilities klasörümüze Security adında bir klasör ekliyoruz. Daha sonra bu klasörümüzü de JWT adında bir klasör ekleyiğ içerisine AccessToken adında bir sınıf ekliyoruz. İçerisine de

```

public string Token { get; set; }
public DateTime Expiration { get; set; }
public int CompanyId { get; set; }

```

Kodlarımızı ekliyoruz.

Daha sonra TokenOptions adında bir class daha ekliyoruz. İçerisine de

```

public string Audience { get; set; }
public string Issuer { get; set; }
public int AccesTokenExpriation { get; set; }
public string SecurityKey { get; set; }

```

Kodlarımızı ekliyoruz.

Daha sonra da ITokenHelper adında bir interfaces ekliyoruz.

```

AccessToken CreateToken(User user, List<OperationClaim> operationClaims, int
companyId);

```

İçersine de bu kodu ekliyoruz.

Burada şu ana kadar genel manada ne yaptığımıza gelecek olursak. Giriş ve login işlemlerimiz için yani Authorize işlemlerimiz için WebApi olduğu için Token kullanacağız. Token çünkü biz giriş yaptığımızda bize kriptolu uzun bir şifre veriyor. O şifrenin de bir geçerlilik süresi oluyor. O geçerlilik süresini kullanıp kullanmamak bizlere bağlı.

AccesToken bu Tokenımızın expiration yani ne kadar süre sistemde kalacağını ve CompanyId yani bizim şirketimizin Id sini tutacak.

TokenOptions ise bizim genel tanımlamalarımız. Zaten token ın default yapısında o şekil tanımlanması gerekiyor. O yapıyı da buraya kurduk ki WebApi katmanında onu tanımlayacağız.

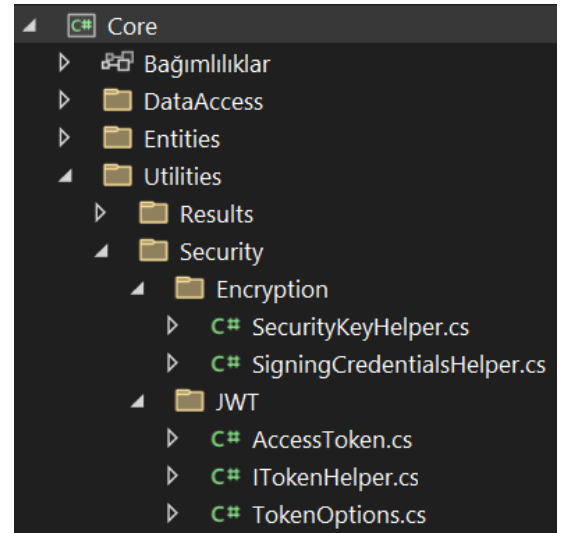
ITokenHelper ise AccesToken için bir CreateToken oluşturması için gereken imzasını yazdık. İmzanın içerisini dolduracağız. Yani metodun içerisine tekrardan bir class yazacağız ve class ın içerisini dolduracağız. Ama class doldurmadan önce bir şifreleme anahtarlarımızı da yazmalıyız.

Bunun içinde Security içerisine tekrardan bir Encryption adında folder ekleyerek başlıyoruz. Bu da bizim token için anahtarlamaları yani kriptolanacak kodlamayı yazacağız yani bize vereceği anahtarı yazacağız. Bu folder içerisine SecurityKeyHelper.cs adında bir class ekliyoruz. İçerisine bir dışarıdan da erişebilelim diye statik yapı kuracağız. Güvenlik anahtarı oluşturabilsin diye kuruyoruz. Birazdan ITokenHelper da çağırırken yani imzanın içerisine metot ile doldururken kullanacağız. İçerisine

```

public static SecurityKey CreateSecurityKey(string securityKey)
{
    return new SymmetricSecurityKey(Encoding.UTF8.GetBytes(securityKey));
}

```



```
}
```

Kodunu ekledik.

Daha sonra SigningCredentialsHelper adında bir class oluşturduk.

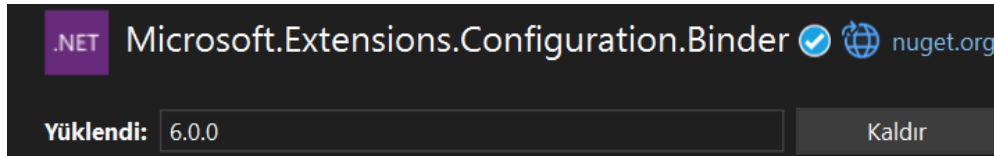
```
public static SigningCredentials CreateSigningCredentials(SecurityKey
securityKey)
{
    return new SigningCredentials(securityKey,
SecurityAlgorithms.HmacSha512Signature);
}
```

İçerisine de bu kodu ekledik.

JwtHelper Yazmak

Şimdi Core katmanımızda Utilities içerisindeki JWT klasörüne JwtHelper adında bir class ekliyoruz. ITokenHelper dan implement alacak. Daha sonra içeriğini doldurduk.

TokenOptions kısmında bir hata alıyoruz bunun için Core katmanına bir paket kurduk.



Bu paketi kurduğumuzda hatamız gitti.

ClaimExtensions Yazmak

Bunun için Core katmanına Extensions (genişletmek) adında bir folder ekliyoruz. İçerisine de ClaimsExtensions adında bir class ekliyoruz. Bu class static yapıda olacak. JwtHelper de private IEnumerable içerisinde tanımladıklarımızı burada tanımlayacağız.

IAuthService

Bunun için Business katmanında Abstract içerisine IAuthService adında bir interface ekliyoruz. Bunu kullanıcı girişi, kaydı, kontrolü için kullanacağız.

IUserService : Kullanıcıya ait yetkileri getirecek ve Mail ile kullanıcıyı getirecek.

UserManager : Business katmanında Concrete kalsörü içerisine UserManager sınıfı ekliyoruz

AuthManager : Business katmanında Concrete kalsörü içerisine AuthManager sınıfı ekliyoruz.

HashingHelper : Core katmanında Utilities klasörüne Hashing adında bir klasör ekliyoruz. Orayada HashingHelper adında bir class ekliyoruz. Bizim göndereceğimiz şifreyi burada salt ile beraber birleştirip bir şifre oluşturacağız. Sonrada Login işleminde de biz şifremizi biz tekrardan alıp tekrardan bir hashleme ve salt ile beraber kriptolu şifre haline dönüştüreceğiz ve harf harf kontrol ettireceğiz. Burada ilk öncelikle bir şifreleme oluşturduk. Daha sonra kriptoladık ve harf harf tek tek kontrol ettirdik.

Program.cs'de Configuration Ayarları

appsettings.json dosyamızda yazdığımız

```
"TokenOptions": {  
  "Audience": "www.rumeys.com",  
  "Issuer": "www.rumeys.com",  
  "AccessTokenExpiration": 60, //Tokenın kaç dakikaya kapanacağını söylüyor.  
  "SecurityKey": "mysecretkeymysecretkey0022"  
}
```

Şu anda bu yazdığımız kodları program tanımayacak bunun için Program.cs dosyasında tanımlayacağız.

//Burası bağlantımızın ayarını yaptığımız yer yani WebApi ye ulaşacak Apileri burada belirliyoruz.

```
builder.Services.AddCors(options =>  
{  
  options.AddPolicy("AllowOrigin",  
    builder => builder.WithOrigins("https://localhost:4200"));  
});
```

WebApi katmanında Register işlemleri

WebApi katmanında Controllers klasörüne AuthController.cs adında API denetleyicisi ekliyoruz.

```
[Route("api/[controller]")]  
[ApiController]
```

Api olarak geçerli olabilmesi için yukarıdaki kod controllerlarda gerekiyor.

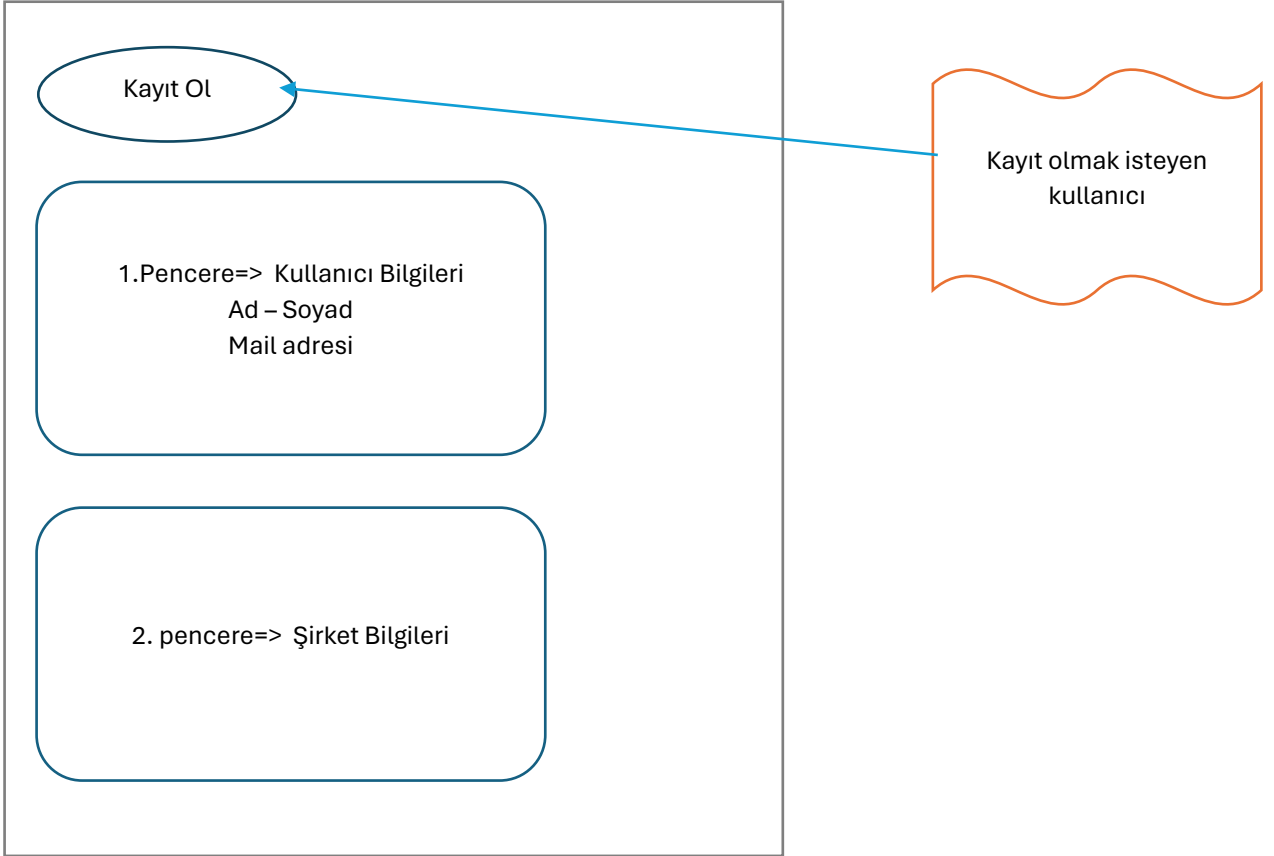
Şu zamana kadar neler yaptık?

Öncelikle Sql üzerinden bir veritabanı oluşturdum. Bu veritabanına da tek tek table yapımı oluşturdum.

Arkasından geldim bir VisualStudio üzerinden bir proje oluşturdum. Bu projeye de katmanlı mimari kullanarak Business, Core, DataAccess, Entities ve WebApi katmanlarımı ekledim. Entities ve DataAccess katmanımda Concrete klasörlerine classlarımı Abstract klasörlerine de interfacerlerimi oluşturdum. Dto da kullanıcı register işlemleri için classlar oluşturdum. Bu yapıları Generic hale getirmek için de Core katmanında DataAccess ile bağlantılı IEntityRepository ile generic yapıya dönüştürdüm. Ardından Core katmanına Utilities içerisinde Security, hashing, result işlemlerim var. Result ben bir işlem yaptığımda bana bir mesaj gönderebilsin diye oluşan bir altyapı. Business katmanında ise Concrete, Abstract dışında DependencyResolvers Autofac teknolojisini kullanarak Autofac yaptım. Aynı zamanda da WebApi katmanında Program.cs içerisinde bu Autofac servisini tanımladım. Ayrıca buranın içerisinde jsWebToken ayarlarını da yapmış oldum.

Kullanıcı ve Şirket Kaydı

Web sitemize bir kullanıcı gelecek (register) kayıt olmak isteyecek bir register butonumuz olacak. Kayıt butonuna bastığında bize birinci pencere açılacak. Bu pencere de bu kullanıcının mail adresini, adını soyadını, ve bilgilerini isteyecek sonra bu kullanıcı devam tuşuna bastığında bu program kaydetmemesi lazım çünkü biz demiştik ki kullanıcılar şirket ile ilişkili yani bir kullanıcı var ise bir şirket ile de ilişkili olması lazım. Bu ikinci pencereye geçecek. İkinci penceremizde de bizden şirket bilgilerini isteyecek. Bu iki pencere de eğer tamam ise bu sefer kayıt işlemini tamamlasın bize bir token vererek sisteme giriş yaptırmış olsun. Angular kısmında token aldıktan sonra otomatik giriş yaptırmasını zaten sağlayacağız.



Bizim iki tane register(kayıtol) işlemimiz olacak. Bu temel seviye bir register işlemi. Yani bir kullanıcı sisteme kayıt olabilecek, kendine ait bir şirket seçebilecektir. Giriş yapıp admin paneline ulaştıktan sonra bizim artık yeni penceremizde artık bu kullanıcı ekstradan kendi şirketine ait yeni bir kullanıcı ekleyebilmesi lazım. Yani şöyle düşünelim bir amir buraya kayıt oluyor ve şirketine ait çalışanları yetkileriyle eklemek istiyor. Onun için burada bu kullanıcılara da kayıt edebileceği bir ekran olması lazım ama orada şunu yapması lazım sen bu kullanıcıyı kayıt et ama aynı zamanda ben sana şirketid ni de gönderiyorum. Otomatik bir şirketid si göndermesi lazım. Onun için biz iki tane register işlemi yapacağız.

Mail Parametrelerini Kayıt Etme (Mail Gönderme İşlemi)
Bunun için Business katmanında IMailParameterService içerisine
Mail parametrelerini kaydetmek için : **IResult Update(MailParameter mailParameter)**
Mail parametrelerini çekmek için: **IDataResult<MailParameter> Get(int companyId);**
Kodlarını ekledik.